

SIMATS ENGINEERING

ASSESSMENT TOOL 1- CO4-AT1 – COMPARATIVE ANALYSIS

NAME: S. Bhuvanesh

REG NO: 192521004

Question 15

Compare GPU bitonic sort (CUDA) versus CPU parallel quick sort (64 cores) for sorting 1 billion integers. Analyze parallel time complexity ($O(\log^2 n)$ vs. $O((n \log n)/p)$), memory hierarchy (global vs. shared memory), warp divergence, and PCIe transfer overhead. For what input sizes does GPU sorting become advantageous?

1. INTRODUCTION

Sorting 1 billion (10^9) 32-bit integers is a memory-bandwidth-bound problem that exposes fundamentally different execution models on a GPU versus a multi-core CPU. GPU bitonic sort exploits thousands of lightweight SIMT threads executing a fixed, data-independent comparison network, while CPU parallel quicksort exploits a small number of heavyweight, cache-coherent cores executing an adaptive, data-dependent divide-and-conquer algorithm. This comparison analyzes both approaches across four dimensions: parallel time complexity, memory hierarchy behaviour, control-flow divergence, and host-device transfer

overhead, and then identifies the crossover point at which GPU execution becomes the faster choice.

2. PARALLEL TIME COMPLEXITY ANALYSIS

2.1 GPU Bitonic Sort — $O(\log^2 n)$

Bitonic sort is a sorting network with $\log_2 n$ merge stages, and each stage requires $\log_2 n$ compare-exchange sub-stages, giving a total of $\log^2 n$ sequential rounds. Within a single round, all $n/2$ compare-exchange operations are independent and can be executed by separate GPU threads simultaneously, so the parallel time complexity is $O(\log^2 n)$ provided the machine supplies $n/2$ processing elements. For $n = 10^9$, $\log_2 n \approx 30$, giving roughly 900 sequential rounds — a very small serial depth compared to the size of the input.

- Work complexity (total operations across all threads): $O(n \log^2 n)$ — higher total work than quicksort's $O(n \log n)$, traded for a much shallower critical path.
- A real GPU (e.g., tens of thousands of CUDA cores) does not have $n/2$ physical processors for $n = 10^9$, so each round is itself tiled across many thread blocks and executed in several waves; the $O(\log^2 n)$ bound is the idealized PRAM-style depth, not the wall-clock time on hardware with finite parallelism.

2.2 CPU Parallel Quicksort — $O((n \log n)/p)$

Parallel quicksort partitions the array recursively and assigns independent sub-array sorts to different cores/threads. With $p = 64$ cores and reasonably balanced partitions, the average-case sequential work $O(n \log n)$ is divided across p cores to

give $O((n \log n)/p)$ parallel time, plus overhead for partitioning, load balancing and the final merge of thread-local results. For $n = 10^9$ and $p = 64$, this is roughly $(10^9 \times 30)/64 \approx 4.7 \times 10^8$ unit operations per core.

- This bound assumes near-perfect load balancing across all recursive calls; skewed pivots (worst case $O(n^2)$ sequential work) can degrade the parallel bound significantly unless a good pivot-selection strategy (e.g., median-of-medians or random sampling) is used.
- Parallelism is limited by the number of physical cores (64) and by the depth of the recursion tree that still has enough independent sub-problems to keep all cores busy; near the leaves of the recursion, sub-arrays become small and parallel efficiency drops.

2.3 Complexity Comparison

Metric	GPU Bitonic Sort	CPU Parallel Quicksort (64 cores)
Parallel time	$O(\log^2 n)$	$O((n \log n)/p)$
Total work	$O(n \log^2 n)$	$O(n \log n)$ average
Processors assumed	$\approx n/2$ (thousands, tiled)	$p = 64$
Dependence on data	Data-independent (oblivious)	Data-dependent (pivot choice matters)

Metric	GPU Bitonic Sort	CPU Parallel Quicksort (64 cores)
Worst case	Same as average (fixed network)	$O(n^2)$ sequential if pivots are poor

The key asymptotic distinction is that bitonic sort's depth grows only logarithmically ($\log^2 n$) regardless of n , while quicksort's parallel depth still grows with n , only divided by a constant factor p . This means GPU depth scales far better as n grows extremely large, but GPU sort does strictly more total work, which only pays off when there is enough parallel hardware to absorb that extra work in the same or less wall-clock time.

3. MEMORY HIERARCHY ANALYSIS

3.1 GPU: Global vs. Shared Memory

For 1 billion integers (4 GB as 32-bit ints), the entire dataset cannot fit in a GPU's on-chip shared memory (typically 48–228 KB per streaming multiprocessor), so early bitonic stages must operate directly on global memory (GDDR6/HBM, hundreds of GB/s bandwidth but high latency, several hundred cycles). An efficient CUDA bitonic sort therefore uses a hybrid strategy:

- Early stages (large stride compare-exchange across distant elements): performed with global-memory reads/writes, coalesced where possible to maximize achieved bandwidth.

- Later stages (once compare-exchange distance shrinks to fit a thread block's tile): the sub-array segment is staged into shared memory (on-chip SRAM, ~19 TB/s effective bandwidth, ~20–30 cycle latency), sorted locally with many bitonic micro-stages, then written back once — dramatically reducing global memory traffic.
- Registers hold per-thread compared elements during each compare-exchange, minimizing shared-memory bank conflicts when indexing is done carefully (e.g., avoiding stride patterns that collide on the same bank).

3.2 CPU: Cache Hierarchy

A 64-core CPU sorts using its cache hierarchy: small per-core L1 (32–64 KB) and L2 (256KB–1MB) caches, and a larger shared L3 (tens of MB). Quicksort's partitioning step scans a contiguous sub-array, which is naturally cache-friendly (good spatial locality) as long as the sub-array fits within L2/L3; but the top-level partitions over the full 4 GB array are inherently memory-bound, streaming from DRAM at DDR bandwidth (tens of GB/s aggregate) — far lower than GPU global-memory bandwidth. As recursion proceeds and sub-arrays shrink to fit in L2/L3, cache-resident partitioning and even an in-cache insertion-sort cutoff become effective, sharply reducing memory traffic per element.

3.3 Comparison

Aspect	GPU	CPU
Fast on-chip memory	Shared memory / registers (small, per-SM)	L1/L2 per-core, shared L3
Bulk data path	Global memory (GDDR6/HBM, high bandwidth)	DRAM via cache hierarchy (lower bandwidth)
Locality strategy	Tile sub-arrays into shared memory once local enough	Recursive partitioning naturally shrinks working set into cache
Bandwidth advantage	Much higher raw global-memory bandwidth	Lower raw bandwidth, but fewer bytes moved per useful comparison at small scale

4. WARP DIVERGENCE (GPU-SPECIFIC)

CUDA executes threads in warps of 32, and all threads in a warp execute the same instruction in lockstep (SIMT). Bitonic sort's comparison network is data-oblivious — the sequence of compare-exchange operations is fixed by index, not by data values — so within a warp, threads typically follow the same control-flow path (compare two fixed indices, conditionally swap using a branchless min/max

or predicated select rather than an if/else swap). This makes bitonic sort largely warp-divergence-free, which is one of its biggest structural advantages on GPU hardware.

- Any divergence that does occur is usually confined to boundary handling (e.g., stride patterns near tile edges) or to branchless swap implementation choices, and is small relative to the main comparison workload.
- Contrast: a naive GPU port of quicksort or a comparison algorithm with true if/else branching per element (e.g., data-dependent pivot comparisons) would cause severe warp divergence, since different threads in the same warp would take different branches on different data values — serializing execution within the warp. This is precisely why quicksort is a poor fit for the GPU's SIMT model and why bitonic sort (or other oblivious networks like odd-even merge sort) is preferred on CUDA.

5. PCIE TRANSFER OVERHEAD

GPU sorting must first move the 1 billion integers (4 GB) from host (CPU) memory to device (GPU) memory over PCIe, and move the sorted result back. On PCIe Gen4 x16 ($\approx 25\text{--}28$ GB/s achievable) this is roughly $4\text{ GB} / 25\text{ GB/s} \approx 160\text{ ms}$ each way, or about 320 ms round-trip — before any sorting computation begins. On PCIe Gen3 x16 ($\sim 12\text{--}15$ GB/s achievable) this overhead roughly doubles.

- This transfer time is pure overhead that the CPU-only approach never pays, since CPU quicksort operates in-place on data already resident in system RAM.

- For the GPU approach to win overall, the compute-time savings from massive on-chip parallelism must exceed this fixed PCIe cost; for small inputs, PCIe transfer alone can dominate total GPU wall-clock time.
- Mitigations in practice: pinned (page-locked) host memory for faster DMA transfers, overlapping transfer with computation via CUDA streams, and — on unified-memory architectures (e.g., NVLink-connected systems or integrated GPUs) — avoiding the discrete PCIe hop altogether.

6. WHEN DOES GPU SORTING BECOME ADVANTAGEOUS?

Combining all four factors gives three regimes based on input size n :

- Small n (roughly below ~ 1 – 10 million elements): PCIe transfer time and kernel-launch/synchronization overhead dominate; the GPU's extra $O(n \log^2 n)$ work and fixed latency costs are not amortized, so 64-core CPU quicksort (already highly parallel and cache-friendly at this scale) is typically faster.
- Medium n (tens of millions to a few hundred million): the two approaches converge; GPU throughput starts to overcome PCIe overhead, but CPU cache-resident partitioning is still efficient, so results depend heavily on the specific GPU/CPU generation, memory bandwidth, and implementation quality.
- Large n (roughly ≥ 500 million–1 billion elements, as in this problem): the CPU's DRAM bandwidth and finite core count (64) become the bottleneck, its cache hierarchy can no longer keep large working sets resident, and the fixed PCIe cost becomes a small fraction of total sort time. The GPU's much higher aggregate global-memory bandwidth, massive thread-level parallelism, and near-divergence-free execution let it dominate; GPU bitonic sort is generally

the faster choice for the full 1-billion-integer workload described in this question.

7. SUMMARY TABLE

Factor	GPU Bitonic Sort (CUDA)	CPU Parallel Quicksort (64 cores)
Parallel time complexity	$O(\log^2 n)$ — shallow, near-constant depth	$O((n \log n)/p)$ — depth still grows with n
Total work	$O(n \log^2 n)$ — more total operations	$O(n \log n)$ average — less total work
Fast memory used	Shared memory / registers for local tiles	L1/L2/L3 cache for shrinking sub-arrays
Bulk memory bandwidth	High (GDDR6/HBM global memory)	Lower (system DRAM)
Control-flow behaviour	Largely divergence-free (oblivious network)	N/A on CPU (branch prediction handles it well)
Extra fixed cost	PCIe host↔device transfer (~hundreds of ms for 4 GB)	None (data already in RAM)
Best for	Very large n (hundreds of millions to billions)	Small–medium n

8. CONCLUSION

For 1 billion integers, GPU Bitonic Sort using CUDA is generally more attractive when the data is already on the GPU or when the PCIe transfer cost can be amortized. Its regular computation, massive thread-level parallelism, high memory bandwidth, and low warp divergence make it suitable for very large datasets.

However, CPU parallel Quick Sort can be preferable for smaller datasets or when the data is already in CPU memory, because it avoids PCIe transfers.

Final comparison

For the given 1-billion-integer problem, the GPU becomes especially advantageous when the sorting workload is large enough that its higher throughput outweighs the CPU-to-GPU transfer overhead.